

# Relatório AgroVision AI

*Tecnologias Emergentes - ADS*

*Ruan da Silva das Neves*

Repositório: [https://github.com/NevesRuan/Yui\\_AgroVision-AI](https://github.com/NevesRuan/Yui_AgroVision-AI)

## Introdução

Todo o código do AgroVision AI foi gerado de forma assistida por IA (agentes de codificação operando sobre instruções estruturadas). O ponto central desta entrega é demonstrar que código gerado por IA funciona, mas não está pronto, ele precisa de revisão humana para se tornar sustentável, seguro e correto.

Cada trecho abaixo segue o formato pedido: o que o código fazia originalmente, qual era o problema, o que foi melhorado e por que a nova versão é superior. Os exemplos são reais e estão refletidos no estado atual do repositório, organizados em três categorias: acoplamento e arquitetura, robustez e correção, e segurança.

## Trechos revisados

### 3.1 - Acoplamento da rota ao cliente HTTP (httpx)

**O que o código fazia originalmente.** A rota /chat, em app.py, capturava diretamente a exceção da biblioteca HTTP usada internamente pelo cliente do Ollama:

```
import httpx
...
try:
    answer = await ollama_client.chat(messages)
except httpx.HTTPError:
    raise HTTPException(status_code=503, detail="Ollama indisponível...")
```

**Qual era o problema encontrado.** A camada de rotas (app.py) conhecia um detalhe de implementação do ollama\_client, o fato de ele usar httpx. Se o cliente trocasse de biblioteca HTTP, o app.py teria que mudar junto. Isso quebra o encapsulamento: a rota não deveria saber como o cliente fala com o Ollama, apenas que a operação pode falhar.

**O que foi melhorado.** Foi criada uma exceção de domínio no próprio cliente, e o app.py passou a capturá-la sem importar httpx:

```
# services/ollama_client.py
class OllamaUnavailableError(RuntimeError):
    """Falha de comunicacao com o Ollama (timeout, conexao recusada, etc)."""

# app.py
from services.ollama_client import OllamaUnavailableError
...
except OllamaUnavailableError as exc:
    logger.warning("Falha na chamada ao Ollama: %s", exc)
    raise HTTPException(status_code=503, detail="Servico de IA temporariamente indisponível")
from exc
```

**Por que a nova versão é melhor.** A dependência do httpx fica isolada em um único arquivo. A rota passa a depender de um conceito do domínio (“o serviço de IA está indisponível”), não de uma biblioteca concreta. Trocar a implementação do transporte HTTP não exige tocar em nenhuma rota.

### 3.2 - Evento de ciclo de vida obsoleto (on\_event)

**O que o código fazia originalmente.** A inicialização e o encerramento da aplicação usavam o decorador `@app.on_event`:

```
@app.on_event("startup")
async def on_startup():
    init_db()
    monitor.start()
    asyncio.create_task(ollama_client.warmup())

@app.on_event("shutdown")
async def on_shutdown():
    monitor.stop()
```

**Qual era o problema encontrado.** `@app.on_event` está deprecated no FastAPI desde a versão 0.93 e emite `DeprecationWarning`. Além disso, a task de warmup era criada sem guardar a referência, o coletor de lixo do Python pode descartá-la antes de concluir, gerando um warmup silenciosamente perdido.

**O que foi melhorado.** Migração para o padrão atual lifespan, com a referência das tasks guardada em `app.state`:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    monitor.start()
    warmup_task = asyncio.create_task(ollama_client.warmup())
    app.state.warmup_task = warmup_task
    weather_task = asyncio.create_task(_persist_weather_loop())
    app.state.weather_task = weather_task
    try:
        yield
    finally:
        monitor.stop()
        for task in (warmup_task, weather_task):
            if not task.done():
                task.cancel()

app = FastAPI(title="AgroVision AI", lifespan=lifespan)
```

**Por que a nova versão é melhor.** Segue o padrão oficial e não emite avisos de obsolescência. Setup e teardown ficam no mesmo bloco, deixando explícito o que precisa ser desligado. As tasks ganham referência guardada e cancelamento ordenado no encerramento, eliminando o risco de coleta prematura e de tasks órfãs após o shutdown.

### 3.3 - Timeout na geração lenta do LLM (CPU-only)

**O que o código fazia originalmente.** A função `chat()` do cliente fazia uma única requisição não-streaming ao Ollama, aguardando a resposta inteira de uma vez:

```
async def chat(messages):
    async with httpx.AsyncClient(timeout=OLLAMA_TIMEOUT) as client:
        r = await client.post(OLLAMA_URL, json=_base_payload(messages, stream=False))
```

```
r.raise_for_status()
return r.json()["message"]["content"]
```

**Qual era o problema encontrado.** Com `stream=False`, o `OLLAMA_TIMEOUT` (120s padrão) valia para a geração inteira. Em máquinas sem GPU rodando llama3 8B, uma análise completa levava até ~214s, acima do timeout, disparando `ReadTimeout` e derrubando respostas legítimas. Aumentar o timeout global mascararia o problema.

**O que foi melhorado.** A `chat()` passou a consumir o streaming internamente, acumulando os chunks:

```
async def chat(messages: list[dict]) -> str:
    parts: list[str] = []
    async for chunk in chat_stream(messages):
        parts.append(chunk)
    return "".join(parts)
```

**Por que a nova versão é melhor.** Consumindo o streaming, o `OLLAMA_TIMEOUT` passa a valer por chunk (intervalo entre tokens) em vez de pela geração completa. Enquanto o modelo emitir tokens, a conexão não expira, mesmo que a resposta inteira leve minutos. O resultado externo continua idêntico, mas a tolerância a geração lenta em hardware modesto melhora drasticamente, sem inflar o timeout global.

### 3.4 - Ausência de limite de tamanho no payload do chat

**O que o código fazia originalmente.** O schema de entrada validava apenas o tamanho mínimo:

```
class ChatRequest(BaseModel):
    question: str = Field(..., min_length=1)
    history: list[ChatMessage] = Field(default_factory=list)
```

**Qual era o problema encontrado.** Não havia limite superior. Um cliente malicioso (ou um bug) poderia enviar uma `question` de 10 MB ou um `history` com dezenas de milhares de mensagens. O Pydantic processa o payload inteiro antes de o handler rodar. Vetor trivial de negação de serviço (DoS) por exaustão de memória/CPU.

**O que foi melhorado.** Limites no schema e uma defesa em profundidade via middleware que rejeita corpos grandes pelo `Content-Length`, antes de carregá-los na memória:

```
# services/schemas.py
class ChatMessage(BaseModel):
    role: Literal["user", "assistant", "system"]
    content: str = Field(..., min_length=1, max_length=4000)

class ChatRequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=2000)
    history: list[ChatMessage] = Field(default_factory=list, max_length=20)

# app.py
@app.middleware("http")
async def limit_body_size(request: Request, call_next):
    content_length = request.headers.get("content-length")
    if content_length is not None:
        declared = int(content_length)
        if declared > MAX_REQUEST_BODY_BYTES:  # 256 KB por padrao
            return JSONResponse({"detail": "Corpo da requisicao muito grande"},
                                status_code=413)
    return await call_next(request)
```

**Por que a nova versão é melhor.** São duas barreiras independentes. O middleware rejeita o corpo (413) antes mesmo de lê-lo na RAM, corta o DoS de memória e o bypass do rate limit por corpos volumosos inválidos. O schema garante limites semânticos por campo (422 com mensagem clara). Um request legítimo continua passando; o ataque é barrado com custo mínimo.

### 3.5 - Mensagens de erro vazando a arquitetura

**O que o código fazia originalmente.** Quando o Ollama estava fora, a API devolvia uma mensagem técnica diretamente ao cliente:

```
detail="Ollama indisponível. Verifique se 'ollama serve' esta rodando."
```

**Qual era o problema encontrado.** A mensagem expõe ao cliente externo exatamente qual stack roda por trás (Ollama, comando 'ollama serve'). Em um ambiente exposto, isso entrega informação de reconhecimento a um atacante, qual software atacar, qual porta procurar. Mensagens de erro devem ser úteis ao operador (nos logs), não ao público.

**O que foi melhorado.** Mensagem genérica para o cliente, detalhe técnico apenas no log do servidor:

```
except OllamaUnavailableError as exc:
    logger.warning("Falha na chamada ao Ollama: %s", exc)    # detalhe no log
    raise HTTPException(
        status_code=503,
        detail="Servico de IA temporariamente indisponível", # generico ao cliente
    ) from exc
```

**Por que a nova versão é melhor.** O cliente recebe o suficiente para entender que deve tentar mais tarde (503), sem pistas sobre a infraestrutura. O operador mantém visibilidade total nos logs. Separa-se a informação por audiência, princípio básico de tratamento seguro de erros.

### 3.6 - Falha ao formatar dado climático ausente (bug)

**O que o código fazia originalmente.** Ao montar o contexto do agente, a temperatura e a umidade eram formatadas diretamente:

```
lines.append(
    f" - Temperatura: {weather.get('temperature_c'):.1f} C, "
    f"umidade {weather.get('humidity_pct')}%"
)
```

**Qual era o problema encontrado.** A camada de scraping valida os dados contra faixas plausíveis e retorna None quando a fonte devolve um valor absurdo. Quando temperature\_c chegava como None, o `:.1f` lançava `TypeError: unsupported format string passed to NoneType.__format__`, derrubando a rota /chat inteira. O mesmo padrão sem proteção existia no frontend. Bug real, reproduzível, identificado em auditoria.

**O que foi melhorado.** Tratamento campo a campo, com texto de fallback no backend e no frontend:

```
# services/monitoring_agent.py
temp = weather.get("temperature_c")
humidity = weather.get("humidity_pct")
temp_txt = f"{temp:.1f} C" if temp is not None else "indisponível"
humidity_txt = f"{humidity}%" if humidity is not None else "indisponível"
lines.append(f" - Temperatura: {temp_txt}, umidade {humidity_txt}")
```

```
// static/dashboard.js
document.getElementById("weather-temp").textContent =
  data.temperature_c != null ? `${data.temperature_c.toFixed(1)} °C` : "-";
```

**Por que a nova versão é melhor.** Uma falha em um dado opcional e externo (clima) deixa de derrubar uma funcionalidade central (o chat com o agente). O sistema degrada com elegância, mostra “indisponível” em vez de quebrar. Validação objetiva: `build_event_context([], {"temperature_c": None, ...})` executa sem exceção.

### 3.7 - Tabela de banco órfã (código morto + incoerência)

**O que o código fazia originalmente.** A camada de scraping criava a tabela `weather_snapshots` e definia a função `save_weather_snapshot(...)`, mas nada no sistema chamava essa função.

**Qual era o problema encontrado.** Infraestrutura criada sem uso: a tabela era inicializada no `init_db()` mas permanecia eternamente vazia. Código morto ou pior, incoerente com a documentação, que afirma persistir o histórico climático. Quem rodasse o sistema encontraria uma tabela vazia, contradizendo o que estava escrito.

**O que foi melhorado.** Uma task periódica no lifespan passou a persistir snapshots de clima de hora em hora, gravando apenas dados frescos:

```
async def _persist_weather_loop():
    from services.event_repository import save_weather_snapshot
    while True:
        try:
            if WEATHER_ENABLED:
                snap = await weather_scraper.get_current()
                if snap is not None and not snap.is_stale:
                    save_weather_snapshot(snap.to_dict())
        except Exception as exc:
            logger.warning("Falha ao persistir clima: %s", exc)
        await asyncio.sleep(3600)
```

**Por que a nova versão é melhor.** O código deixa de ter uma função e uma tabela sem propósito. A persistência ganha utilidade real, histórico climático para correlacionar com os eventos detectados pelo YOLO. E o sistema passa a fazer o que a documentação promete. Grava apenas snapshots frescos, evitando duplicar dados velhos.

### 3.8 - Sombreamento de variável

**O que o código fazia originalmente.** O nome `events` era usado tanto para a função-rota quanto para variáveis locais em outras rotas.

```
@app.get("/events")
async def events():
    ...

@app.post("/chat")
async def chat(...):
    events = list_events(AGENT_EVENT_LIMIT)  # mesmo nome da funcao-rota
```

**Qual era o problema encontrado.** A variável local `events` sombreava a função `events` no escopo do módulo. Não causava erro em runtime, mas é um code smell: linters acusam, e em refatorações futuras (mover rotas para um `APIRouter`) o conflito pode gerar bugs sutis difíceis de rastrear.

**O que foi melhorado.** A variável local foi renomeada para `recent_events`:

```
@app.post("/chat")
async def chat(request: Request, payload: ChatRequest):
    recent_events = list_events(AGENT_EVENT_LIMIT)
    messages = build_agent_messages(payload.question, payload.history, recent_events,
    weather_dict)
```

**Por que a nova versão é melhor.** Elimina a ambiguidade léxica e melhora a expressividade, o nome `recent_events` comunica que são eventos recentes, não a função que os busca. Remove warnings de análise estática. Custo praticamente zero, ganho de legibilidade e segurança em refatoração.

### 3.9 - Código morto (`capture_store`)

**O que o código fazia originalmente.** O módulo `services/capture_store.py` definia `list_captures(...)`, que listava as imagens salvas das detecções. A função existia e funcionava, mas nenhuma rota a expunha.

**Qual era o problema encontrado.** Código morto. Um módulo inteiro carregado e mantido sem ser usado por nenhum caminho do sistema. Código morto aumenta a superfície de manutenção, confunde quem lê o projeto e tende a apodrecer (deixa de acompanhar mudanças no resto do código).

**O que foi melhorado.** O módulo foi removido por completo. Confirmação: `grep -rn "capture_store|list_captures"` retorna vazio.

**Por que a nova versão é melhor.** Menos código para manter, ler e entender. A regra adotada foi explícita: ou a funcionalidade é usada (exposta por uma rota), ou é removida, não se mantém infraestrutura “por via das dúvidas”. Coerência com a decisão tomada em 3.7 para o caso oposto.

### 3.10 - Cabeçalhos de segurança HTTP ausentes

**O que o código fazia originalmente.** As respostas HTTP saíam sem nenhum cabeçalho de proteção, comportamento padrão do FastAPI, herdado do código gerado.

**Qual era o problema encontrado.** Sem `X-Content-Type-Options`, `X-Frame-Options` e `Referrer-Policy`, a aplicação fica exposta a MIME sniffing, clickjacking (embutir o dashboard em um iframe malicioso) e vazamento de URL via cabeçalho `Referer`. São proteções básicas que toda aplicação web servida a um navegador deveria ter.

**O que foi melhorado.** Um middleware adiciona os cabeçalhos em todas as respostas:

```
@app.middleware("http")
async def security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers.setdefault("X-Content-Type-Options", "nosniff")
    response.headers.setdefault("X-Frame-Options", "DENY")
    response.headers.setdefault("Referrer-Policy", "no-referrer")
    return response
```

**Por que a nova versão é melhor.** Defesa em profundidade aplicada de forma centralizada (um único ponto, vale para todas as rotas). `nosniff` impede o navegador de reinterpretar o tipo de conteúdo; `DENY` bloqueia clickjacking; `no-referrer` evita vazar a URL interna. Melhoria proativa, além das apontadas na revisão, o tipo de hardening que o código gerado por IA não traz por padrão.

## Conclusão

Os dez trechos acima ilustram um padrão consistente: o código gerado por IA funcionava, subia, respondia, detectava, conversava, mas carregava fragilidades que só uma leitura crítica revela: acoplamentos que dificultariam a evolução, casos de borda não tratados que viravam bugs reais, e lacunas de segurança invisíveis enquanto o sistema apenas funciona.

Nenhum desses problemas aparecia ao executar o sistema pela primeira vez. Todos exigiram revisar o código como produto, não como exercício. É a lição central da disciplina: a IA acelera a escrita, mas o julgamento técnico, sobre arquitetura, segurança e correção, permanece responsabilidade do desenvolvedor. A diferença entre código que roda e código sustentável foi, em cada caso, uma decisão humana de revisão.